

WebSocket 双向流式-V3

豆包语音

法律声明

本《豆包语音》的所有内容，包括但不限于文字、商标、架构、图示、图片、页面布局等，其知识产权（著作权、商标权、专利权、商业秘密等）归属于北京火山引擎科技有限公司及其关联公司（火山引擎），非经火山引擎书面同意，任何个人和组织不得复制、使用、修改、转发或以任何违反本《豆包语音》所承载的目的进行传播。

本《豆包语音》陈述内容仅作为产品的通用性介绍和参考性指引，火山引擎保留按“现状”和“当前可用”的形式提供产品和服务的权利。火山引擎不对本《豆包语音》中所载的产品功能、性质、质量、标准等内容进行明示或默示的保证和承诺，最终以您与火山引擎实际签署的协议为准。

如您发现本《豆包语音》有任何错误或歧义，或发现有对本《豆包语音》、产品本身的侵权行为，请与火山引擎取得联系。

联系方式：service@volcengine.com，400-850-0030
(周一至周五 10:00-18:00)

目录

法律声明-----1

目录-----1

1. WebSocket 双向流式-V3-----1

1. WebSocket 双向流式-V3

语音合成大模型API列表

根据具体场景选择合适的语音合成大模型API。

接口	推荐场景	接口功能	文档链接
<code>wss:// openspeech.bytedance.com/api/v3/tts/bidirection</code>	WebSocket协议，实时交互场景，支持文本实时流式输入，流式输出音频。	语音合成、声音复刻、混音	V3 WebSocket双向流式文档
<code>wss:// openspeech.bytedance.com/api/v3/tts/unidirectional/stream</code>	WebSocket协议，一次性输入合成文本，流式输出音频。	语音合成、声音复刻、混音	V3 WebSocket单向流式文档
<code>https:// openspeech.bytedance.com/api/v3/tts/unidirectional</code>	HTTP Chunked协议，一次性输入全部合成文本，流式输出音频。	语音合成、声音复刻、混音	V3 HTTP Chunked单向流式文档
<code>https:// openspeech.bytedance.com/api/v3/tts/unidirectional/sse</code>	HTTP SSE协议，一次性输入全部合成文本，流式输出音频。	语音合成、声音复刻、混音	V3 Server Sent Events (SSE) 单向流式文档

1 接口功能

双向流式API为用户提供文本转语音能力，支持多语种、多方言，支持WebSocket协议流式调用，同时支持一包发送请求数据或者边发边收数据的流式交互方式。

1.1 最佳实践

该接口会处理碎片化的文本或者过长的文本，整理为长度合适的句子。因此最大的优势是平衡延迟和合成效果。

在对接大文本模型时，推荐将流式输出的文本直接输入该接口，而不要额外增加切句或者攒句的逻辑。同样的文本调用一次该接口与多次调用合成接口相比，前者会更为自然，情绪更饱满。

推荐使用链接复用的方式接入，在每次使用时只需要建立一次websocket连接，即发送startconnection，在收到ConnectionStarted后，即为链接建立成功。此时，发送startsession，通过taskrequest发送文本，同时接收音频。在没有文本可以发送时，需立即发送finish session。如果还有文本发送需要合成，此时无需

断开链接，待收到SessionFinished后（必须），重新发送startsession，开启新一轮session。如果再没有文本需要合成，即可发送finish connection断开链接。具体流程可参考该页交互示例**。**
注意，同一个WebSocket链接下支持多次session，但不支持同时多个session。

2 接口说明

2.1 请求Request

请求路径

· 服务使用的请求路径：`wss://openspeech.bytedance.com/api/v3/tts/bidirection`

建连&鉴权

鉴权Request Headers

在 websocket 建连的 HTTP 请求头（Request Header 中）添加以下信息
使用[新版控制台](#)时，推荐采用以下更简化的鉴权方式。

Key	说明	参数类型	是否必须	Value示例
X-API-Key	使用火山引擎控制台获取的API Key，可参考 控制台API Key管理	string	必须	"your-api-key"
X-API-Resource-Id	表示调用服务的资源信息 ID，可以用来选择不同的模型版本效果，也决定了计费方式。	string	必须	豆包语音合成大模型 语音合成接口通过 X-API-Resource-Id 参数来选择不同的版本效果： · <code>seed-tts-2.0</code> 仅支持调用"豆包语音合成模型 2.0"的音色 · <code>seed-tts-1.0 / seed-tts-1.0-concurr</code> 仅支持调用"豆包语音合成模型 1.0"的音色

Key	说明	参数类型	是否必须	Value示例
				同时， X-Api-Resource-Id 也决定了计费方式： · seed-tts-2.0 ：对应计费商品为“语音合成2.0字符版” · seed-tts-1.0 ：对应计费商品为“语音合成1.0字符版” · seed-tts-1.0-concurr ：对应计费商品为“声音复刻1.0并发版” 豆包声音复刻大模型 语音合成接口通过 X-Api-Resource-Id 参数来选择不同的版本效果： · seed-icl-2.0 ：对应声音复刻2.0版本效果 · seed-icl-1.0 / seed-icl-1.0-concurr ：对应声音复刻1.0版本效果 同时， X-Api-Resource-Id 也决定了计费方式： · seed-icl-2.0 ：对应计费商品为“声音复刻2.0字符版”

Key	说明	参数类型	是否必须	Value示例
				· <code>seed-icl-1.0</code> : 对应计费商品为“声音复刻1.0 字符版” · <code>seed-icl-1.0-concurr</code> : 对应计费商品为“声音复刻1.0 并发版”
X-Api-Connect-Id	用于追踪当前连接情况的标志 ID 建议用户传递，便于排查连接情况 该session id不可复用，每个 session需保证ID 是唯一的（ 请求失败重连的情况 session id也需重新生成，不可复用上一次session 的ID ）	string	可选	“67ee89ba-7050-4c04-a3d7-ac61a63499b3”

Python

```
headers = {  
    "X-Api-Key": "your-api-key",  
    "X-Api-Resource-Id": "seed-tts-2.0"  
}
```

若使用[旧版控制台](#)，鉴权方式如下。建议尽快切换至新版，以体验更便捷的鉴权流程。

Key	说明	参数类型	是否必须	Value示例
X-Api-App-Id	使用火山引擎控制台获取的APP ID，可参考 控制台使用FAQ-Q1 🔗 （旧版控制	string	必须	“123456789”

Key	说明	参数类型	是否必须	Value示例
	台使用，新版控制台只需要X-Api-Key即可)			
X-Api-Access-Key	使用火山引擎控制台获取的 Access Token，可参考 控制台使用FAQ-Q1 (旧版控制台使用，新版控制台只需要X-Api-Key即可)	string	必须	“your-access-key”
X-Api-Resource-Id	表示调用服务的资源信息 ID，可以用来选择不同的模型版本效果，也决定了计费方式。	string	必须	豆包语音合成大模型 语音合成接口通过 X-Api-Resource-Id 参数来选择合适的版本效果： - seed-tts-2.0 仅支持调用“豆包语音合成模型 2.0”的音色 - seed-tts-1.0 / seed-tts-1.0-concurr 仅支持调用“豆包语音合成模型 1.0”的音色 同时，X-Api-Resource-Id 也决定了计费方式： - seed-tts-2.0 ：对应计费商品为“语音合成 2.0字符版 ” - seed-tts-1.0 ：对应计费商品

Key	说明	参数类型	是否必须	Value示例
				为“语音合成1.0字符版” · <code>seed-tts-1.0-concurr</code>：对应计费商品为“声音复刻1.0并发版” 豆包声音复刻大模型 语音合成接口通过 <code>X-API-Resource-Id</code> 参数来选择不同的版本效果： · <code>seed-icl-2.0</code>：对应声音复刻2.0版本效果 · <code>seed-icl-1.0 / seed-icl-1.0-concurr</code>：对应声音复刻1.0版本效果 同时，<code>X-API-Resource-Id</code> 也决定了计费方式： · <code>seed-icl-2.0</code>：对应计费商品为“声音复刻2.0字符版” · <code>seed-icl-1.0</code>：对应计费商品为“声音复刻1.0字符版” · <code>seed-icl-1.0-concurr</code>：对应计费商品为“声音复刻1.0并发版”

Key	说明	参数类型	是否必须	Value示例
X-Api-Connect-Id	用于追踪当前连接情况的标志 ID 建议用户传递，便于排查连接情况 该session id不可复用，每个 session需保证ID 是唯一的（ 请求失败重连的情况 session id也需重新生成，不可复用上一次session 的ID ）	string	可选	“67ee89ba-7050-4c04-a3d7-ac61a63499b3”

Python

```
headers = {  
    "X-Api-App-Id": "123456789",  
    "X-Api-Access-Key": "your-access-key",  
    "X-Api-Resource-Id": "seed-tts-2.0"  
}
```

额外Request Headers

Key	说明	是否必须	Value示例
X-Control-Require-Usage-Tokens-Return	请求消耗的用量返回控制标记。当携带此字段，在 SessionFinish事件（ 152 ）中会携带用量数据	否	· 设置为*，表示返回已支持的用量数据。 · 也设置为具体的用量数据标记，如 text_words；多个用逗号分隔 · 当前已支持的用量数据 ◦ text_words，表示计费字符数

Response Headers

在 websocket 握手成功后，会返回这些 Response header

Key	说明	Value 示例
X-Tt-Logid	服务端返回的 logid，建议用户获取和打印方便定位问题	202407261553070FACFE6D19421815D605

WebSocket 二进制协议

WebSocket 使用二进制协议传输数据。
协议的组成由至少 4 个字节的可变 header、payload size 和 payload 三部分组成，其中

- header 描述消息类型、序列化方式以及压缩格式等信息
- 可选字段
 - event 字段，用于描述连接过程中状态管理的预定义事件
 - connect id size/ connect id 字段，用于描述连接类事件的额外信息
 - session id size/ session id 字段，用于描述会话类事件的额外信息
 - error code: 仅用于错误帧，描述错误信息
- payload size 是 payload 的长度
- payload 是具体负载内容，依据消息类型不同 payload 内容不同。

需注意：协议中整数类型的字段都使用**大端**表示。

二进制帧

Byte	Left 4-bit	Right 4-bit	说明
0 – Left half	Protocol version		目前只有v1，始终填 0b0001
0 – Right half		Header size (4x)	目前只有4字节，始终填 0b0001
1	Message type	Message type specific flags	下文详细说明
2 – Left half	Serialization method		· 0b0000 : Raw（无特殊序列化方式，主要针对二进制音频数据）

Byte	Left 4-bit	Right 4-bit	说明
			· 0b0001 : JSON (主要针对文本类型消息)
2 – Right half		Compression method	· 0b0000 : 无压缩 · 0b0001 : gzip
3	Reserved		留空 (0b0000 0000)
[4 ~ 7]	[Optional field,like event number,...]		取决于Message type specific flags, 可能有、也可能没有
...	Payload		可能是音频数据、文本数据、音频文本混合数据

Message type & specific flags

Message type & specific flags是重点扩展的部分，详细说明如下：

Message type	含义	Message type specific flags	是否包含Event number	备注
0b0001	Full-client request	0b0100	是	完整请求体，用于触发服务端 session初始化
0b1001	Full-server response	0b0100	是	TTS：前端信息、文本音频混合数据等 (Serialization=JSON)
0b1011	Audio-only response	0b0100	是	

Message type	含义	Message type specific flags	是否包含Event number	备注
0b1111	Error information	None	否	

Payload 请求参数

注意：TTS服务参数设置仅在StartSession时生效，每次发送的文本在TaskRequest时生效。
TTS服务参数具体如下：

字段	描述	是否必须	类型	默认值
user	用户信息			
user.uid	用户uid			
event	请求的事件	✓		
namespace	请求方法		string	BidirectionalTTS
req_params.text	required，输入文本（双向流式不支持ssml）	✓	string	
req_params.model	模型版本，传 <code>seed-tts-1.1</code> 较默认版本音质有提升，并且延时更优，不传为默认效果。 注：若使用1.1模型效果，在复刻场景中会放大训练音频prompt特质，因此对prompt的要求更高，使用高质量的训练音频，可		string	

字段	描述	是否必须	类型	默认值
	以获得更优的音质效果。 以下参数仅针对声音复刻2.0的音色生效，即音色ID的前缀为 <code>saturn_</code> 的音色。音色的取值为以下两种： <code>seed-tts-2.0-expressive</code>：表现力较强，支持QA和Cot能力，不过可能存在抽卡的情况。 <code>seed-tts-2.0-standard</code>：表现力上更加稳定，但是不支持QA和Cot能力。如果此时使用QA或Cot能力，则拒绝请求。 - 如果不传 <code>model</code> 参数，默认使用 <code>seed-tts-2.0-expressive</code> 模型。			
req_params.speaker	发音人，具体见 发音人列表	✓	string	
req_params.audio_params	音频参数，便于服务节省音频解码耗时	✓	object	
req_params.audio_params.format	音频编码格式，mp3/ogg_opus/pcm。接口传入wav并不会报		string	mp3

字段	描述	是否必须	类型	默认值
	错，在流式场景下传入wav会多次返回wav header，这种场景建议使用pcm。			
req_params.audio_params.sample_rate	音频采样率，可选值 [8000,16000,22050,24000,32000,44100,48000]		number	24000
req_params.audio_params.bit_rate	音频比特率，可传16000、32000等。 bit_rate默认设置范围为64k ~ 160k，传了disable_default_bit_rate为true后可以设置到64k以下 GoLang示例： additions = fmt.Sprintf("{\"disable_default_bit_rate\":true}") 注：针对MP3和ogg格式建议主动设置bit_rate，若使用默认值(实际被设置为8k)会出现音质损耗比较严重的情况；wav计算比特率跟pcm一样是 比特率 (bps) = 采样率 × 位深度 × 声道数； 目前大模型TTS只能改采样率，所以对于wav格式来说只能通过		number	

字段	描述	是否必须	类型	默认值
	改采样率来变更音频的比特率；			
req_params.audio_params.emotion	设置音色的情感。示例: "emotion": "angry" 注：当前仅部分音色支持设置情感，且不同音色支持的情感范围存在不同。 详见： 大模型语音合成API-音色列表-多情感音色 		string	
req_params.audio_params.emotion_scale	调用emotion设置情感参数后可使用emotion_scale进一步设置情绪值，范围1~5，不设置时默认值为4。 注：理论上情绪值越大，情感越明显。但情绪值1~5实际为非线性增长，可能存在超过某个值后，情绪增加不明显，例如设置3和5时情绪值可能接近。		number	4
req_params.audio_params.speech_rate	语速，取值范围[-50,100]，100代表2.0倍速，-50代表0.5倍数		number	0
	音量，取值范围[-50,100]，100		number	0

字段	描述	是否必须	类型	默认值
req_params.audio_params.loudness_rate	代表2.0倍音量，-50代表0.5倍音量（mix音色暂不支持）			

字段	描述	是否必须	类型	默认值
req_params.audio_params.enable_timestamp	设置 "enable_timestamp": true 返回句级别字的时间戳（默认为 false，参数传入 true 即表示启用）开启后，在原有返回的事件 <code>event=TTSSentenceEnd</code> 中，新增该子句的时间戳信息。 · 一个子句的时间戳返回之后才会开始返回下一句音频。 · 合成有多个子句会多次返回 <code>TTSSentenceStart</code> 和 <code>TTSSentenceEnd</code>。开启字幕后字幕跟随 <code>TTSSentenceEnd</code> 返回。 · 字/词粒度的时间戳，其中字/词是tn。具体可以看下面的例子。 · 支持中、英，其他语种、方言暂时不支持。 注：该参数只在TTS1.0("豆包语音合成模型1.0"的音色)、ICL1.0生效。		bool	false

字段	描述	是否必须	类型	默认值
req_params.audio_params.enable_subtitle	设置 "enable_subtitle" : true 返回句级别字的时间戳（默认为 false，参数传入 true 即表示启用）开启后，新增返回事件 <code>event=TTSSubtitle</code>，包含字幕信息。 · 在一句音频合成之后，不会立即返回该句的字幕。合成进度不会被字幕识别阻塞，当一句的字幕识别完成后立即返回。可能一个子句的字幕返回的时候，已经返回下一句的音频帧给调用方了。 · 合成有多个子句，仅返回一次 <code>TTSSentenceStart</code> 和 <code>TTSentenceEnd</code>。开启字幕后会多次返回 <code>TTSubtitle</code>。 · 字/词粒度的时间戳，其中字/词是原文。具体可以看下面的例子。 · 支持中、英，其他语种、方言暂时不支持；		bool	false

字段	描述	是否必须	类型	默认值
	· latex公式不支持 ◦ req_params.additions.enable_latex_tn为true时，不开启字幕识别功能，即不返回字幕； · ssml不支持 ◦ req_params.ssml 不传时，不开启字幕识别功能，即不返回字幕； 注：该参数只在TTS2.0、ICL2.0生效。			
req_params.additions	用户自定义参数		jsonstring	
req_params.additions.silence_duration	设置该参数可在句尾增加静音时长，范围0~30000ms。（注：增加的句尾静音主要针对传入文本最后的句尾，而非每句话的句尾）		number	0
req_params.additions.enable_language_detector	自动识别语种		bool	false

字段	描述	是否必须	类型	默认值
req_params.additions.disable_markdown_filter	是否开启markdown解析过滤，为true时，解析并过滤markdown语法，例如， <code>**你好**</code> ，会读为“你好”，为false时，不解析不过滤，例如， <code>**你好**</code> ，会读为“星星‘你好’星星”		bool	false
req_params.additions.disable_emoji_filter	开启emoji表情在文本中不过滤显示，默认为false，建议搭配时间戳参数一起使用。 GoLang示例： <pre>additions = fmt.Sprintf("{\"disable_emoji_filter\":true}")</pre>		bool	false
req_params.additions.mute_cut_remain_ms	该参数需配合mute_cut_threshold参数一起使用，其中： "mute_cut_threshold": "400", // 静音判断的阈值（音量小于该值时判定为静音） "mute_cut_remain_ms": "50", // 需要保留的静音长度 注：参数和value都为string格式 Golang示例：		string	

字段	描述	是否必须	类型	默认值
	additions = fmt.Sprintf("{\"mute_cut_threshold\":\"400\", \"mute_cut_remain_ms\": \"1\"}") 特别提醒： · 因MP3格式的特殊性，句首始终会存在100ms内的静音无法消除，WAV格式的音频句首静音可全部消除，建议依照自身业务需求综合判断选择 · "豆包语音合成模型2.0"的音色 🔗 暂不支持 · 豆包声音复刻模型2.0（icl 2.0）的音色暂不支持			
req_params.additions.enable_latex_tn	是否可以播报latex公式，需将disable_markdown_filter设为true		bool	false
req_params.additions.latex_parser	是否使用lid 能力播报latex公式，相较于latex_tn 效果更好； 值为“v2”时支持lid能力解析公式，值为“”时不支持lid； 需同时将disable_markdo		string	

字段	描述	是否必须	类型	默认值
	wn_filter设为true;			
req_params.additions.max_length_to_filter_parenthesis	是否过滤括号内的部分，0为不过滤，100为过滤		int	100
req_params.additions.explicit_language（明确语种）	仅读指定语种的文本 精品音色和 声音复刻ICL 1.0场景： · 不给定参数，正常中英混 · <code>crosslingual</code> 启用多语种前端（包含 <code>zh/en/ja/es-mx/id/pt-br</code>） · <code>zh-cn</code> 中文为主，支持中英混 · <code>en</code> 仅英文 · <code>ja</code> 仅日文 · <code>es-mx</code> 仅墨西哥 · <code>id</code> 仅印尼 · <code>pt-br</code> 仅巴葡 DIT 声音复刻场景： 当音色是使用 <code>model_type=2</code>训练的，即采用dit标准版效果时，建议指定明确语种，目前支持： · 不给定参数，启用多语种前端 <code>zh,en,ja,es-</code>		string	

字段	描述	是否必须	类型	默认值
	mx,id,pt-br,de,fr · zh,en,ja,es-mx,id,pt-br,de,fr 启用多语种前端 · zh-cn 中文为主，支持中英混 · en 仅英文 · ja 仅日文 · es-mx 仅墨西哥 · id 仅印尼 · pt-br 仅巴葡 · de 仅德语 · fr 仅法语 当音色是使用model_type=3训练的，即采用dit还原版效果时，必须指定明确语种，目前支持： · 不给定参数，正常中英混 · zh-cn 中文为主，支持中英混 · en 仅英文 声音复刻 ICL2.0 场景： 当音色是使用model_type=4训练的 · 不给定参数，正常中英混 · zh-cn 中文为主，支持中英混 · en 仅英文			

字段	描述	是否必须	类型	默认值
	GoLang示例: additions = fmt.Sprintf("{\"explicit_language\": \"zh\"}")			
req_params.additions.context_language (参考语种)	给模型提供参考的语种 · 不给定 西欧语种采用英语 · id 西欧语种采用印尼 · es 西欧语种采用墨西 · pt 西欧语种采用巴葡		string	
req_params.additions.explicit_dialect (明确方言)	明确方言，目前仅 zh_female_vv_uranus_bigts 音色支持以下三种方言： · dongbei (东北话) · shaanxi (陕西话) · sichuan (四川话) 参数情况举例说明： 1. speaker_id = zh_female_xiaohe_uranus_bigts , explicit_language不传，explicit_dialect=dongbei, 则报参数错		string	

字段	描述	是否必须	类型	默认值
	误，即语种和方言不对应 2. speaker_id = zh_female_vv_uranus_big tts , explicit_language不传，explicit_dialect=dongbei，则正常完成东北方言的合成 3. speaker_id = zh_female_vv_uranus_big tts , explicit_language=ja，explicit_dialect=dongbei，则报参数错误，即语种和方言不对应 4. speaker_id = zh_female_vv_uranus_big tts , explicit_language=ja，explicit_dialect不传，则按照语种正常合成			
req_params.additions.unsupported_char_ratio_thresh	默认: 0.3，最大值: 1.0 检测出不支持合成的文本超过设置的比例，则会返回错误。		float	0.3
req_params.additions.aigc_watermark	默认: false 是否在合成结尾增加音频节奏标识		bool	false
req_params.additions.aigc_metad	在合成音频header加入元数		object	

字段	描述	是否必须	类型	默认值
ata (meta 水印)	据隐式表示，支持 mp3/wav/ogg_opus			
req_params.additions.aigc_metadata.enable	是否启用隐式水印		bool	false
req_params.additions.aigc_metadata.content_producer	合成服务提供者的名称或编码		string	""
req_params.additions.aigc_metadata.produce_id	内容制作编号		string	""
req_params.additions.aigc_metadata.content_propagator	内容传播服务提供者的名称或编码		string	""
req_params.additions.aigc_metadata.propagate_id	内容传播编号		string	""
req_params.additions.cache_config (缓存相关参数)	开启缓存，开启后合成相同文本时，服务会直接读取缓存返回上一次合成该文本的音频，可明显加快相同文本的合成速率，缓存数据保留时间1小时。 (通过缓存返回的数据不会附带时间戳) Golang示例: <pre>additions =</pre>		object	

字段	描述	是否必须	类型	默认值
	<pre>fmt.Sprintf("{\"disable_default_bit_rate\":true,\n\"cache_config\":{\n\"text_type\":1,\n\"use_cache\":true}}")</pre>			
req_params.additions.cache_config.text_type (缓存相关参数)	和use_cache参数一起使用，需要开启缓存时传1		int	1
req_params.additions.cache_config.use_cache (缓存相关参数)	和text_type参数一起使用，需要开启缓存时传true		bool	true
req_params.additions.cache_config.use_segment_cache (缓存相关参数)	和text_type参数一起使用，需要开启缓存时传true 分句文本的缓存，双向流式场景首包耗时比使用use_cache低。		bool	true
req_params.additions.post_process	后处理配置 Golang示例： <pre>additions =\nfmt.Sprintf("{\"post_process\":{\"pitch\":12}}")</pre>		object	
req_params.additions.post_process.pitch	音调取值范围是[-12,12]		int	0

字段	描述	是否必须	类型	默认值
req_params.additions.context_texts (仅TTS2.0支持 🔗)	语音合成的辅助信息，用于模型对话式合成，能更好的体现语音情感； 可以探索，比如常见示例有以下几种： 1. 语速调整 a. 比如： context_texts: ["你可以说慢一点吗? "] 2. 情绪/语气调整 a. 比如： context_texts=["你可以用特别特别痛心的语气说话吗?"] b. 比如： context_texts=["嗯，你的语气再欢乐一点"] 3. 音量调整 a. 比如： context_texts=["你嗓门再小点。"] 4. 音感调整 a. 比如： context_texts=["你能用骄傲的语气来说话吗? "]		string list	null

字段	描述	是否必须	类型	默认值
	注意： - 该字段仅适用于"豆包语音合成模型 2.0"的音色 🔗 - 当前字符串列表只第一个值有效 - 该字段文本不参与计费			
req_params.additions.section_id (仅TTS2.0支持 🔗)	其他合成语音的会话id(session_id)，用于辅助当前语音合成，提供更多的上下文信息；取值，参见接口交互中的session_id示例： - section_id="bf5b5771-31cd-4f7a-b30c-f4ddcbf2f9da" 注意： - 该字段仅适用于"豆包语音合成模型 2.0"的音色 🔗 - 历史上下文的session_id有效期： - 最长30轮 - 最长10分钟		string	""

字段	描述	是否必须	类型	默认值
req_params.additions.use_tag_parser	是否开启cot解析能力。cot能力可以辅助当前语音合成，对语速、情感等进行调整。 注意： - 音色支持范围：仅限声音复刻2.0复刻的音色 - 文本长度：单句的text字符长度最好小于64（cot标签也计算在内） - cot能力生效的范围是单句 示例： 支持单组和多组cot标签：<cot text=急促难耐>工作占据了生活的绝大部分</cot>，只有去做自己认为伟大的工作，才能获得满足感。<cot text=语速缓慢>不管生活再苦再累，都绝不放弃寻找</cot>。		bool	false
[req_params.mix_speaker	混音参数结构 注意： 该字段仅适用于"豆包语音合成模型 1.0"的音色 🔗		object	

字段	描述	是否必须	类型	默认值
req_params.mix_speaker.speakers	混音音色名以及影响因子列表 注意： 1. 最多支持3个音色混音 2. 音色风格差异较大的两个音色（如男女混），以0.5-0.5同等比例混合时，可能出现偶发跳变，建议尽量避免 3. 使用Mix能力时， req_params.speaker = custom_mix_big tts		list	null
req_params.mix_speaker.speakers[i].source_speaker	混音源音色名 注意： 1. 支持"豆包语音合成模型1.0"的音色☞、"语音合成（小模型）"的音色☞、声音复刻大模型的音色 2. 使用声音复刻大模型音色时，使用 <code>s_</code> 开头的 <code>speakerid</code> ，或者使用查询接口获取的 <code>id_</code> 的 <code>speakerid</code> ，不支持 <code>DiT_</code> 或者 <code>saturn_</code>		string	""

字段	描述	是否必须	类型	默认值
	开头的 speakerid			
req_params.mix_speaker.speakers[i].mix_factor	混音源音色名影响因子 注意： 1. 混音影响因子和必须=1		float	0

单音色请求参数示例：

JSON

```
{
  "user": {
    "uid": "12345"
  },
  "event": 100,
  "req_params": {
    "text": "明朝开国皇帝朱元璋也称这本书为万物之根",
    "speaker": "zh_female_shuangkuaisisi_moon_big tts",
    "audio_params": {
      "format": "mp3",
      "sample_rate": 24000
    },
  },
}
```

mix请求参数示例：

```
{
  "user": {
    "uid": "12345"
  },
  "req_params": {
    "text": "明朝开国皇帝朱元璋也称这本书为万物之根",
    "speaker": "custom_mix_big tts",
    "audio_params": {
      "format": "mp3",
```

```
JSON  "sample_rate": 24000
      },
      "mix_speaker": {
        "speakers": [{
          "source_speaker": "zh_male_bvlazysheep",
          "mix_factor": 0.3
        }, {
          "source_speaker": "BV120_streaming",
          "mix_factor": 0.3
        }, {
          "source_speaker": "zh_male_ahu_conversation_wvae_big tts",
          "mix_factor": 0.4
        }
      ]
    }
  }
}
```

2.2 响应Response

建连响应

主要关注建连阶段 HTTP Response 的状态码和 Body

- 建连成功：状态码为 200
- 建连失败：状态码不为 200，Body 中提供错误原因说明

WebSocket 传输响应

文本帧

- 主要通过文本内容反馈异常错误信息

二进制帧

- 主要通过协议约定的方式，结构化返回正常响应和一般错误信息

正常响应帧

Byte	Left 4-bit	Right 4-bit	说明
0 – Left half	Protocol version		目前只有v1，始终填0b0001

Byte	Left 4-bit	Right 4-bit	说明
0 – Right half		Header size (4x)	目前只有4字节，始终填 0b0001
1	Message type	Message type specific flags	下文详细说明
2 – Left half	Serialization method		· 0b0000 : Raw（无特殊序列化方式，主要针对二进制音频数据） · 0b0001 : JSON（主要针对文本类型消息）
2 – Right half		Compression method	· 0b0000 : 无压缩 · 0b0001 : gzip
3	Reserved		留空（ 0b0000 0000 ）
[4 ~ 7]	[Optional field,like event number,...]		取决于Message type specific flags，可能有、也可能没有
...	Payload		可能是音频数据、文本数据、音频文本混合数据

Payload 响应参数

字段	描述	类型	默认值
data	返回的二进制数据包	[]byte	
event	返回的事件类型	number	

字段	描述	类型	默认值
res_params.text	经文本分句后的句子	string	.

错误响应帧

Byte	Left 4-bit	Right 4-bit	说明
0 – Left half	Protocol version		目前只有v1，始终填 0b0001
0 – Right half		Header size (4x)	目前只有4字节，始终填 0b0001
1	Message type	Message type specific flags	固定为 0b11110000
2 – Left half	Serialization method		· 0b0001 : JSON (主要针对文本类型消息)
2 – Right half		Compression method	· 0b0000 : 无压缩
3	Reserved		留空 (0b0000 0000)
[4 ~ 7]	Error code		错误码
...	Payload		错误消息对象

2.3 Event 定义

在 TTS 场景中，Event 是正常数据帧（包括上行和下行）的必要字段，事件定义了请求过程中必要的状态转移。具体的使用过程详见交互示例部分

Event code	含义	事件类型	应用阶段：上行/下行
1	StartConnection, Websocket 阶段申明 创建连接 (在 HTTP 建连 Upgrade 后)	Connect 类	上行
2	FinishConnection, 结 束连接	Connect 类	上行
50	ConnectionStarted, 成 功建连	Connect 类	下行
51	ConnectionFailed, 建 连失败	Connect 类	下行
52	ConnectionFinished 结束连接成功	Connect 类	下行
100	StartSession, Websocket 阶段申明 创建会话	Connect 类	上行
101	CancelSession, 取消 会话 (上行)	Session 类	上行
102	FinishSession, 声明 结束会话 (上行)	Session 类	上行
150	SessionStarted, 成功 开始会话	Session 类	下行
151	SessionCanceled, 已 取消会话	Session 类	下行

Event code	含义	事件类型	应用阶段：上行/下行
152	SessionFinished，会话已结束（上行&下行）	Session 类	下行
153	SessionFailed，会话失败	Session 类	下行
200	TaskRequest，传输请求内容	数据类	上行
350	TTSSentenceStart，TTS 返回句内容开始	数据类	下行
351	TTSSentenceEnd，TTS 返回句内容结束	数据类	下行
352	TTSResponse，TTS 返回句的音频内容	数据类	下行

2.4 时间戳句子格式说明

	TTS1.0 ICL1.0	TTS2.0 ICL2.0
事件交互区别	合成有多个子句会多次返回 TTSSentenceStart 和 TTSSentenceEnd 。开启字幕后字幕跟随 TTSSentenceEnd 返回。	合成有多个子句，仅返回一次 TTSSentenceStart 和 TTSSentenceEnd 。开启字幕后会多次返回 TTSSubtitle 。
返回时机	一个子句的时间戳返回之后才会开始返回下一句音频。	在一句音频合成之后，不会立即返回该句的字幕。合成进度不会被字幕识别阻塞，当一句的字幕识别完成后立即返回。可能一个子句的字幕返回的时

	TTS1.0 ICL1.0	TTS2.0 ICL2.0
		候，已经返回下一句的音频帧给调用方了。
句子返回格式	字幕信息是基于tn打轴	字幕信息是基于原文打轴
	说明 1. text字段对应于：原文 2. words内文本字段对应于：tn 第一句： <pre>{ "phonemes": [], "text": "2019年1月8 日，软件2.0版本于格萨 拉彝族乡应时而生。发 布会当日，一场瑞雪将 天地映衬得纯净无 瑕。", "words": [{ "confidence": 0.8766515, "endTime": 0.295, "startTime": 0.155, "word": "二" }, { "confidence": 0.95224416, "endTime": 0.425, "startTime": 0.295, "word": "零" }, {</pre>	说明 1. text字段对应于：原文 2. words内文本字段对应于：原文 第一句： <pre>{ "phonemes": [], "text": "2019年1月8 日，软件2.0版本于格萨 拉彝族乡应时而生。", "words": [{ "confidence": 0.11120544, "endTime": 0.615, "startTime": 0.585, "word": "2019" }, { "confidence": 0.8413397, "endTime": 0.845, "startTime": 0.615, "word": "年" }, { "confidence": 0.2413961, "endTime": 0.875,</pre>

	TTS1.0 ICL1.0	TTS2.0 ICL2.0
	"confidence": 0.9108828, "endTime": 0.575, "startTime": 0.425, "word": "—" } { "confidence": 0.9609025, "endTime": 0.755, "startTime": 0.575, "word": "九" } { "confidence": 0.96244556, "endTime": 1.005, "startTime": 0.755, "word": "年" } { "confidence": 0.85796577, "endTime": 1.155, "startTime": 1.005, "word": "—" } { "confidence": 0.8460129, "endTime": 1.275, "startTime": 1.155, "word": "月" } { "confidence": 0.90833753, "endTime": 1.505, "startTime": 1.275, "word": "八"	"startTime": 0.845, "word": "1" } { "confidence": 0.8487973, "endTime": 1.055, "startTime": 0.875, "word": "月" } { "confidence": 0.509697, "endTime": 1.225, "startTime": 1.165, "word": "8" } { "confidence": 0.9516253, "endTime": 1.485, "startTime": 1.225, "word": "日， " } ... { "confidence": 0.6933777, "endTime": 5.435, "startTime": 5.325, "word": "而" } { "confidence": 0.921702, "endTime": 5.695, // 第一句结束时 间

	TTS1.0 ICL1.0	TTS2.0 ICL2.0
	<pre>JSON }, { "confidence": 0.9403977, "endTime": 1.935, "startTime": 1.505, "word": "日， " }, ... { "confidence": 0.9415791, "endTime": 10.505, "startTime": 10.355, "word": "无" }, { "confidence": 0.903162, "endTime": 10.895, // 第一句结束时间 "startTime": 10.505, "word": "瑕。 " }] }</pre> 第二句： <pre>{ "phonemes": [], "text": "这仿佛一则自然寓言：我们致力于在</pre>	<pre>JSON "startTime": 5.435, "word": "生。 " }] }</pre> 第二句： <pre>{ "phonemes": [], "text": "发布会当日，一场瑞雪将天地映衬得纯净无瑕。", "words": [{ "confidence": 0.7016578, "endTime": 6.3550415, "startTime": 6.2150416, // 第二句开始时间，是相对整个session的位置 "word": "发" }, { "confidence": 0.6800497, "endTime": 6.4450417, "startTime": 6.3550415, "word": "布" }, ... {</pre>

	TTS1.0 ICL1.0	TTS2.0 ICL2.0
	不断的版本迭代中，为 您带来如雪后初霁般清 晰、焕然一新的体 验。", "words": [{ "confidence": 0.8970245, "endTime": 11.6953745, "startTime": 11.535375, // 第二句开 始时间，是相对整个 session的位置 "word": "这" }, { "confidence": 0.86508185, "endTime": 11.875375, "startTime": 11.6953745, "word": "仿" }, { "confidence": 0.73354065, "endTime": 12.095375, "startTime": 11.875375, "word": "佛" }, { "confidence": 0.8525295, "endTime": 12.275374, "startTime": 12.095375, "word": "—" }... 	JSON "confidence": 0.8818264, "endTime": 10.145041, "startTime": 9.945042, "word": "净" }, { "confidence": 0.87248623, "endTime": 10.285042, "startTime": 10.145041, "word": "无" }, { "confidence": 0.8069703, "endTime": 10.505041, "startTime": 10.285042, "word": "瑕。" }] }

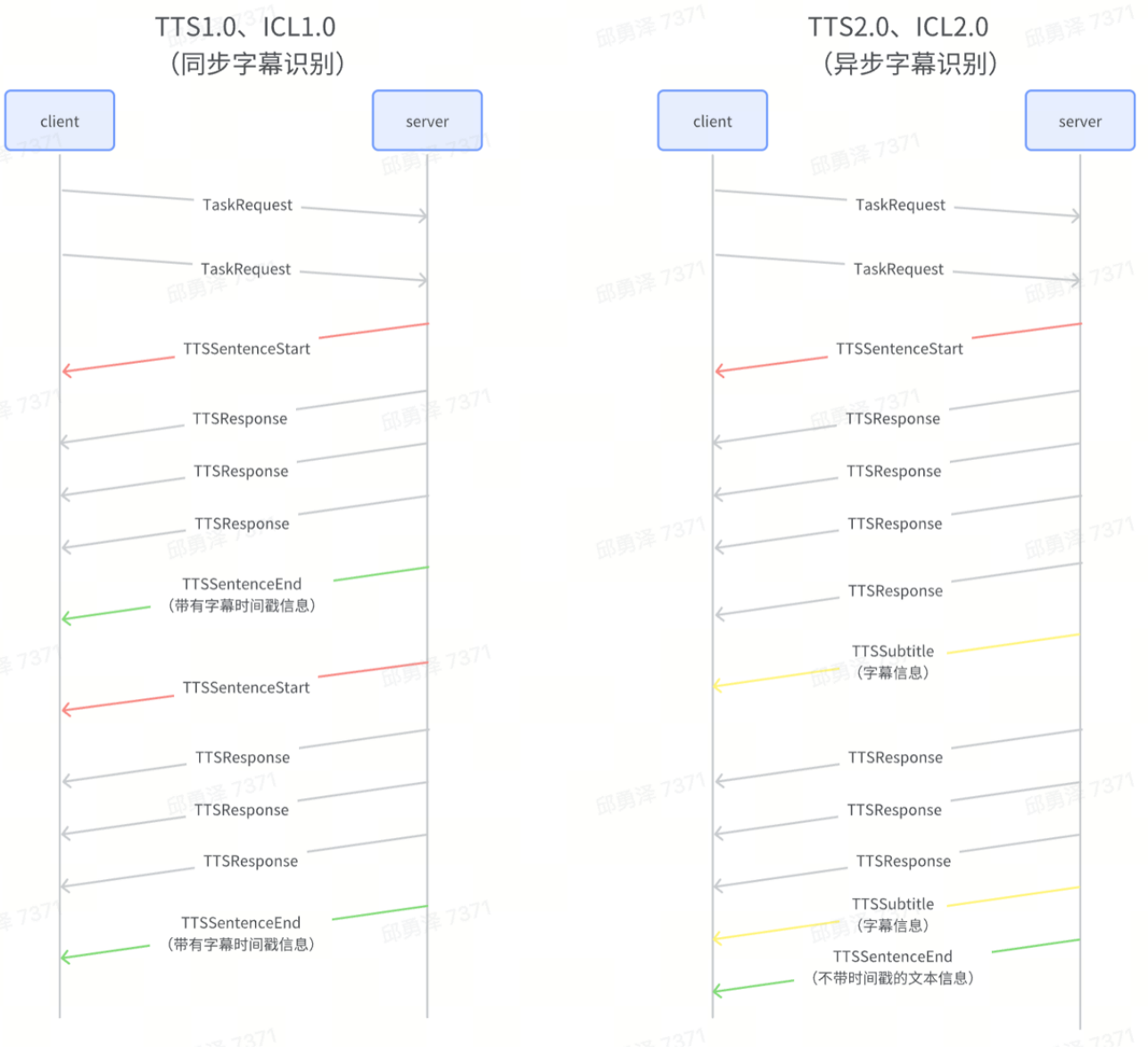
	TTS1.0 ICL1.0	TTS2.0 ICL2.0
	JSON }	
语种	中、英，不支持小语种、方言	中、英，不支持小语种、方言
latex	enable_latex_tn=true，有字幕 返回	enable_latex_tn=true，无字幕 返回，接口不报错
ssml	req_params.ssml不为空，有字 幕返回	req_params.ssml不为空，无字 幕返回，接口不报错

交互示例

TTS1.0、ICL1.0交互

TTS2.0、ICL2.0交互

注：连接建立和断开的部分相同，下图省略。



CancelSession 注意事项:

- 1. `CancelSession` 支持客户端主动放弃当前session，结束合成，并且释放服务端资源。
- 2. `CancelSession` 包发送的最佳时机：收到`SessionStarted`后，发送`FinishSession`之前。
- 3. 客户端在收到 `SessionCanceled` 包之后，如果想要继续合成，需要重新创建session，即重新执行 `StartSession`。

Connection 类:

- `StartConnection` 包 (`RequestMeta`) :

Byte	Left 4-bit	Right 4-bit	备注	
0	0001	0001	v1	4-byte header
1	0001	0100	Full-client request	with event number
2	0001	0000	JSON	no compression
3	0000	0000		
4 – 7	int32(Event_StartConnection)		event type	
8 – 11	uint32(2)		len(<payload_json>)	
12 – 93	JSON { }		payload_json 扩展保留，暂留空JSON	

· FinishConnection 包 (RequestMeta) :

Byte	Left 4-bit	Right 4-bit	备注	
0	0001	0001	v1	4-byte header
1	0001	0100	Full-client request	with event number
2	0001	0000	JSON	no compression

Byte	Left 4-bit	Right 4-bit	备注	
3	0000	0000		
4 – 7	int32(Event_FinishConnection)		event type	
8–11	uint32(2)		len(payload_json)	
12–13	JSON { }		payload_json 扩展保留，暂留空JSON	

· ConnectionStarted 包：

Byte	Left 4-bit	Right 4-bit	备注	
0	0001	0001	v1	4-byte header
1	1001	0100	Full-server response	with event number
2	0001	0000	JSON	no compression
3	0000	0000		
4 – 7	int32(Event_ConnectionStarted)		event type	
8 – 11	uint32(7)		len(<connection_id>)	
12 – 18	bxnweiu		connection_id	

Byte	Left 4-bit	Right 4-bit	备注
19 – 22	uint32(2)		len(payload_json)
23 – 24	JSON { }		payload_json 扩展保留，暂留空JSON

允许客户端不填 connection_id ，由网关下发一个唯一的ID

· ConnectionFailed 包 (ResponseMeta) :

Byte	Left 4-bit	Right 4-bit	备注	
0	0001	0001	v1	4-byte header
1	1001	0100	Full-server response	with event number
2	0001	0000	JSON	no compression
3	0000	0000		
4 – 7	int32(Event_ConnectionFailed)		event type	
8 – 11	uint32(7)		len(<connection_id>)	
12 – 15	uint32(58)		len(<response_meta_json>)	

Byte	Left 4-bit	Right 4-bit	备注
16 - 73	JSON<pre>{ "status_code": 4xxxxxxx, "message": "unauthorized" }</pre>		response_meta_json · 既可能是客户端错误，又可能是服务端错误 · 仅含status_code和message字段

Session 类：

- StartSession 包 (RequestMeta)：

Byte	Left 4-bit	Right 4-bit	备注	
0	0001	0001	v1	4-byte header
1	0001	0100	Full-client request	with event number
2	0001	0000	JSON	no compression
3	0000	0000		
4 - 7	int32(Event_StartSession)		event type	
8 - 11	uint32(12)		len(<session_id>)	
12 - 23	nxckjoejnkegf		session_id	
24 - 27	uint32(...)		len(tts_session_meta)	

Byte	Left 4-bit	Right 4-bit	备注
28 - ...	JSON <pre>{ "user": ..., "req_params": ... }</pre>		tts_session_meta

不允许客户端不填 session_id

· FinishSession 包 (RequestMeta) :

Byte	Left 4-bit	Right 4-bit	备注	
0	0001	0001	v1	4-byte header
1	0001	0100	Full-client request	with event number
2	0001	0000	JSON	no compression
3	0000	0000		
4 - 7	int32(Event_FinishSession)		event type	
8 - 11	int32(12)		len(<session_id>)	
12 - 23	nxckjoejnkegf		session_id	
24 - 27	uint32(2)		len(payload_json)	

Byte	Left 4-bit	Right 4-bit	备注
28 – 29	JSON { }		payload_json 扩展保留，暂留空JSON

· CancelSession 包 (RequestMeta) :

Byte	Left 4-bit	Right 4-bit	备注	
0	0001	0001	v1	4-byte header
1	0001	0100	Full-client request	with event number
2	0001	0000	JSON	no compression
3	0000	0000		
4 – 7	int32(Event_CancelSession)		event type	
8 – 11	int32(12)		len(<session_id>)	
12 – 23	nxckjoejnkegf		session_id	
24 – 27	uint32(2)		len(payload_json)	

Byte	Left 4-bit	Right 4-bit	备注
28 – 29	JSON { }		payload_json 扩展保留，暂留空JSON

· SessionStarted 包 (ResponseMeta) :

Byte	Left 4-bit	Right 4-bit	备注	
0	0001	0001	v1	4-byte header
1	1001	0100	Full-server response	with event number
2	0001	0000	JSON	no compression
3	0000	0000		
4 – 7	int32(Event_SessionStarted)		event type	
8 – 11	uint32(12)		len(<session_id>)	
12 – 23	nxckjoejnkegf		session_id	
24 – 27	uint32(2)		len(payload_json)	

Byte	Left 4-bit	Right 4-bit	备注
28 – 29	JSON { }		payload_json 扩展保留，暂留空JSON

· SessionFinished 包 (ResponseMeta) :

Byte	Left 4-bit	Right 4-bit	备注	
0	0001	0001	v1	4-byte header
1	1001	0100	Full-server response	with event number
2	0001	0000	JSON	no compression
3	0000	0000		
4 – 7	int32(Event_SessionFinished)		event type	
8 – 11	uint32(12)		len(<session_id>)	
12 – 23	nxckjoejnkegf		session_id	
24 – 27	uint32(48)		len(<response_meta_json>)	

Byte	Left 4-bit	Right 4-bit	备注
28 – 75	JSON <pre>{ "status_code": 20000000, "message": "ok", "usage": { "text_words": 4 } }</pre>		response_meta_json · 含status_code和message字段 · usage内容仅在X-Control-Require-Usage-Tokens-Return 激活时返回

- SessionFailed 包 (ResponseMeta) : 与 SessionFinished 类似
- SessionCanceled 包 (ResponseMeta) : 与 SessionFinished 类似

数据类：

- 音频，含Event（以上行 Event_TaskRequest 事件为例）：

Byte	Left 4-bit	Right 4-bit	备注	
0	0001	0001	v1	4-byte header
1	0010	0100	Audio-only request	with event number
2	0000	0000	raw	no compression
3	0000	0000		
4 – 7	int32(Event_TaskRequest)		event type	
8 – 11	uint32(12)		len(<session_id>)	

Byte	Left 4-bit	Right 4-bit	备注
12 - 23	nxckjoejnkegf		session_id
24 - 27	uint32(...)		len(<audios_binary>)
28 - ...	...		audio_binary

· 音频，不含Event（以上行音频为例）：

Byte	Left 4-bit	Right 4-bit	备注	
0	0001	0001	v1	4-byte header
1	0010	0000	Audio-only request	no event number
2	0000	0000	raw	no compression
3	0000	0000		
8 - 11	uint32(12)		len(<session_id>)	
12 - 23	nxckjoejnkegf		session_id	
24 - 27	uint32(...)		len(<audios_binary>)	
28 - ...	...		audio_binary	

· 文本，含Event（以上行 Event_TaskRequest 事件为例）：

Byte	Left 4-bit	Right 4-bit	备注	
0	0001	0001	v1	4-byte header
1	0001	0100	Full-client request	with event number
2	0001	0000	JSON	no compression
3	0000	0000		
4 - 7	int32(Event_TaskRequest)		event type	
8 - 11	uint32(12)		len(<session_id>)	
12 - 23	nxckjoejnkegf		session_id	
24 - 27	uint32(...)		len(<payload_json>)	
28 - ...	{...}		payload_json	

4 错误码

新框架错误码

JSON

CodeOK Code = 20000000 //成功

CodeClientError Code = 45000000 //客户端通用错误

CodeServerError Code = 55000000 //服务端通用错误

CodeSessionError Code = 55000001 //服务端session错误

CodeInvalidReqError Code = 45000001 //客户端请求参数错误

5 调用示例

Python调用示例

前提条件

- 调用之前，您需要获取以下信息：
 - `<appid>`：使用控制台获取的APP ID，可参考 [控制台使用FAQ-Q1](#)。
 - `<access_token>`：使用控制台获取的Access Token，可参考 [控制台使用FAQ-Q1](#)。
 - `<voice_type>`：您预期使用的音色ID，可参考 [大模型音色列表](#)。

Python环境

- Python：3.9版本及以上。
- Pip：25.1.1版本及以上。您可以使用下面命令安装。

Bash

```
python3 -m pip install --upgrade pip
```

下载代码示例

【附件下载】：[volcengine_bidirection_demo.tar.gz](#)，大小为

解压缩代码包，安装依赖

Bash

```
mkdir -p volcengine_bidirection_demo
tar xvfz volcengine_bidirection_demo.tar.gz -C ./volcengine_bidirection_demo
cd volcengine_bidirection_demo
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install --upgrade pip
pip3 install -e .
```

发起调用

`<appid>` 替换为您的APP ID。
`<access_token>` 替换为您的Access Token。
`<voice_type>` 替换为您预期使用的音色ID，例如 `zh_female_cancan_mars_big tts`。

Bash

```
python3 examples/volcengine/bidirection.py --appid <appid> --access_token <access_token> --voice_type <voice_type> --text "你好，我是火山引擎的语音合成服务。这是一个美好的旅程。"
```

Java调用示例

前提条件

- 调用之前，您需要获取以下信息：
 - `<appid>`：使用控制台获取的APP ID，可参考 [控制台使用FAQ-Q1](#)。
 - `<access_token>`：使用控制台获取的Access Token，可参考 [控制台使用FAQ-Q1](#)。
 - `<voice_type>`：您预期使用的音色ID，可参考 [大模型音色列表](#)。

Java环境

- Java：21版本及以上。
- Maven：3.9.10版本及以上。

下载代码示例

【附件下载】：[volcengine_bidirection_demo.tar.gz](#)，大小为

解压缩代码包，安装依赖

Bash

```
mkdir -p volcengine_bidirection_demo
tar xvfz volcengine_bidirection_demo.tar.gz -C ./volcengine_bidirection_demo
cd volcengine_bidirection_demo
```

发起调用

`<appid>` 替换为您的APP ID。
`<access_token>` 替换为您的Access Token。
`<voice_type>` 替换为您预期使用的音色ID，例如 `zh_female_cancan_mars_bigts`。

Bash

```
mvn compile exec:java -Dexec.mainClass=com.speech.volcengine.Bidirection -DappId=<appid> -DaccessToken=<access_token> -Dvoice=<voice_type> -Dtext="**你好**，我是豆包语音助手，很高兴认识你。这是一个愉快的旅程。"
```

Go调用示例

前提条件

- 调用之前，您需要获取以下信息：
 - `<appid>`：使用控制台获取的APP ID，可参考 [控制台使用FAQ-Q1](#)。
 - `<access_token>`：使用控制台获取的Access Token，可参考 [控制台使用FAQ-Q1](#)。
 - `<voice_type>`：您预期使用的音色ID，可参考 [大模型音色列表](#)。

Go环境

- Go：1.21.0版本及以上。

下载代码示例

【附件下载】：[volcengine_bidirection_demo.tar.gz](#)，大小为

解压缩代码包，安装依赖

Bash

```
mkdir -p volcengine_bidirection_demo
tar xvfz volcengine_bidirection_demo.tar.gz -C ./volcengine_bidirection_demo
cd volcengine_bidirection_demo
```

发起调用

`<appid>` 替换为您的APP ID。
`<access_token>` 替换为您的Access Token。
`<voice_type>` 替换为您预期使用的音色ID，例如 `zh_female_cancan_mars_bigts`。

Bash

```
go run volcengine/bidirection/main.go --appid <appid> --access_token <access_token> --
voice_type <voice_type> --text "***你好**，我是火山引擎的语音合成服务。"
```

C#调用示例

前提条件

- 调用之前，您需要获取以下信息：
 - `<appid>`：使用控制台获取的APP ID，可参考 [控制台使用FAQ-Q1](#)。
 - `<access_token>`：使用控制台获取的Access Token，可参考 [控制台使用FAQ-Q1](#)。
 - `<voice_type>`：您预期使用的音色ID，可参考 [大模型音色列表](#)。

C#环境

- .Net 9.0版本。

下载代码示例

【附件下载】: [volcengine_bidirection_demo.tar.gz](#)，大小为

解压缩代码包，安装依赖

Bash

```
mkdir -p volcengine_bidirection_demo
tar xvfz volcengine_bidirection_demo.tar.gz -C ./volcengine_bidirection_demo
cd volcengine_bidirection_demo
```

发起调用

<appid> 替换为您的APP ID。

<access_token> 替换为您的Access Token。

<voice_type> 替换为您预期使用的音色ID，例如 `zh_female_cancan_mars_bigts`。

Bash

```
dotnet run --project Volcengine/Bidirection/Volcengine.Speech.Bidirection.csproj -- --appid
<appid> --access_token <access_token> --voice_type <voice_type> --text "***你好**，这是一个测试文本。我们正在测试文本转语音功能。"
```

TypeScript调用示例

前提条件

- 调用之前，您需要获取以下信息：
 - <appid>：使用控制台获取的APP ID，可参考 [控制台使用FAQ-Q1](#)。
 - <access_token>：使用控制台获取的Access Token，可参考 [控制台使用FAQ-Q1](#)。
 - <voice_type>：您预期使用的音色ID，可参考 [大模型音色列表](#)。

node环境

- node: v24.0版本及以上。

下载代码示例

【附件下载】: [volcengine_bidirection_demo.tar.gz](#)，大小为

解压缩代码包，安装依赖

Bash

```
mkdir -p volcengine_bidirection_demo
tar xvf volcengine_bidirection_demo.tar.gz -C ./volcengine_bidirection_demo
cd volcengine_bidirection_demo
npm install
npm install -g typescript
npm install -g ts-node
```

发起调用

<appid> 替换为您的APP ID。

<access_token> 替换为您的Access Token。

<voice_type> 替换为您预期使用的音色ID，例如 <voice_type> 。

Bash

```
npx ts-node src/volcengine/bidirection.ts --appid <appid> --access_token <access_token> --
voice_type <voice_type> --text "***你好**，我是火山引擎的语音合成服务。"
```